Instituto Politécnico

Escuela Superior de Cómputo

ESCOM

Carrera:

Ingeniería en Sistemas Computacionales

Academia

Sistemas Digitales

Unidad de aprendizaje:

Internet de las cosas|Embedded

Nombre Tarea

Práctica 1 - Mosquitto MQTT con Arduino UNO Q y ESP32

Alumno

Rodríguez Cejudo Marlon Francois

2022630549

Profesor

Miguel Angel Alemán Arce

Índice

Introducción 2

Descripción 2

Instrucciones 2

- Instalación del bróker Mosquitto en Raspberry Pi 2
- Prueba básica del bróker 3
- **Configuración del Publisher en ESP32** 4
- Evidencia de funcionamiento 7

Introducción

Descripción

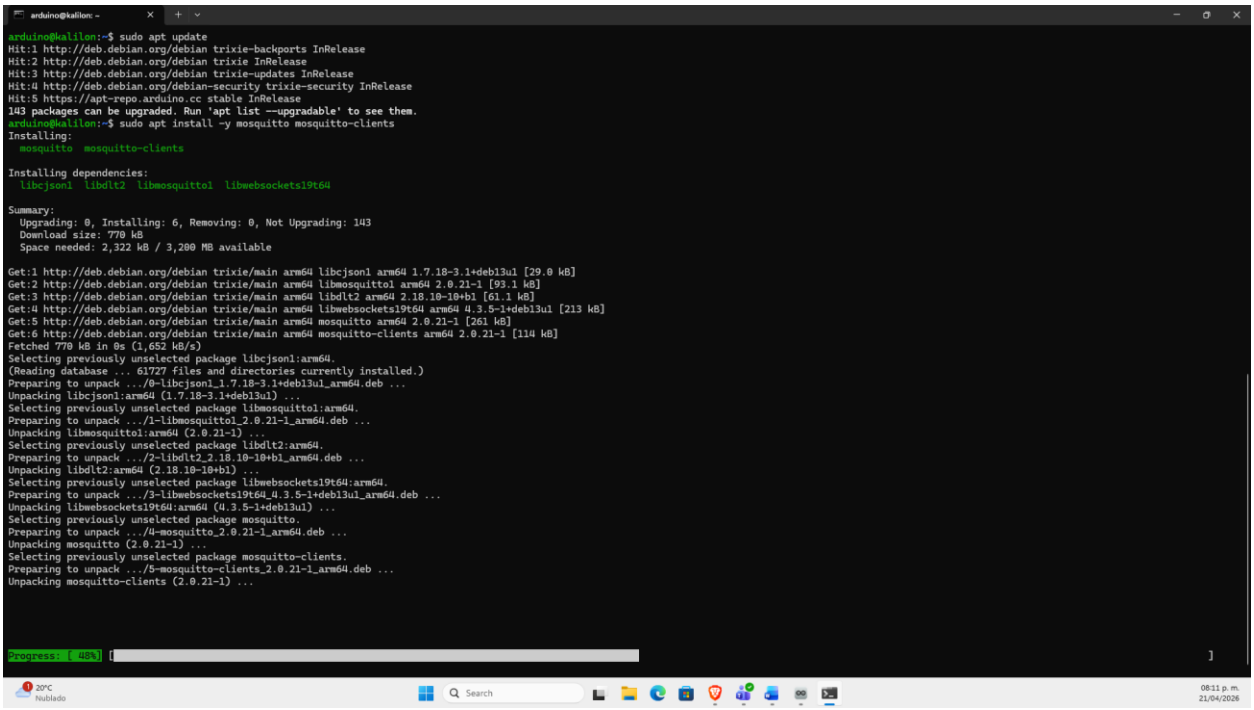
En esta actividad aprenderás a instalar y configurar Mosquitto MQTT en una Raspberry Pi, funcionando como servidor MQTT (broker). Posteriormente, deberás registrar un Publisher utilizando una ESP32, que enviará mensajes al broker. El objetivo es que comprendas cómo funciona el protocolo MQTT en arquitecturas IoT y cómo se integran dispositivos embebidos con servidores ligeros de mensajería.

Instrucciones

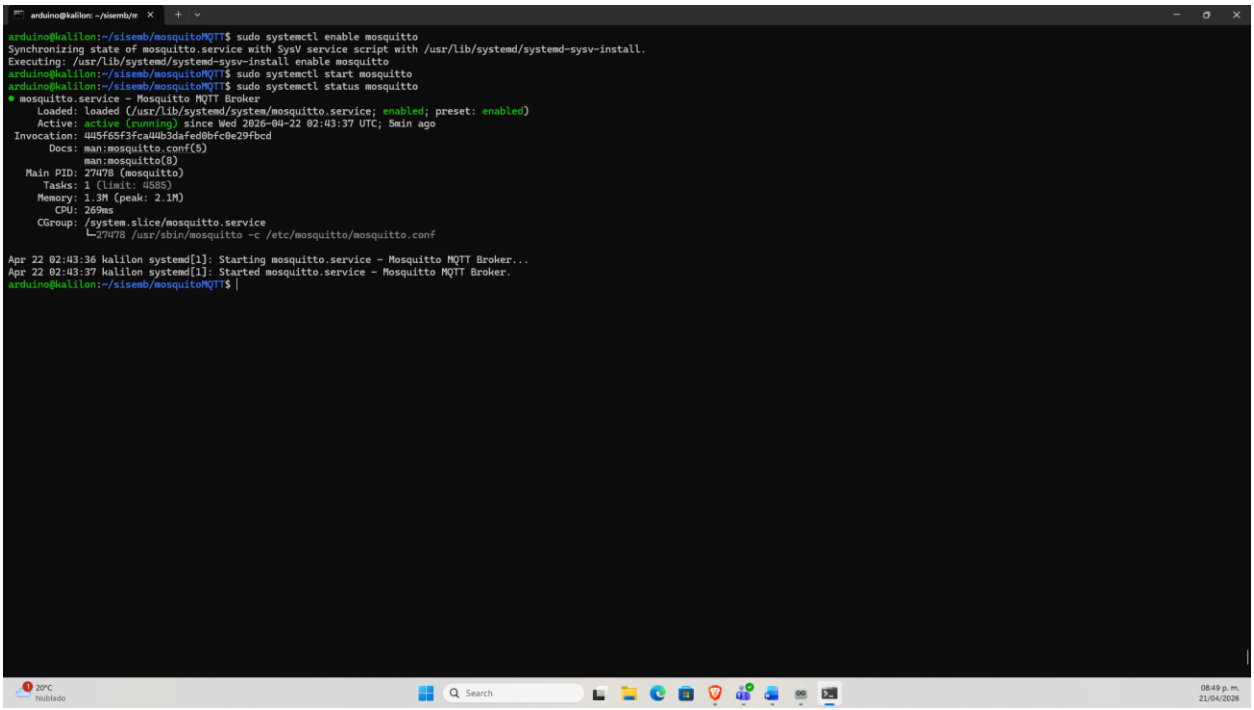
Instalación del bróker Mosquitto en Raspberry Pi

Para esta practica se utilizó la Arduino Uno Q, en lugar de la Raspberry Pi, esta placa contiene un microprocesador y un microcontrolador, y el sistema operativo que utiliza es Debian.

Para esta practica empezaremos actualizando los paquetes y luego instalamos Mosquitto junto con sus herramientas

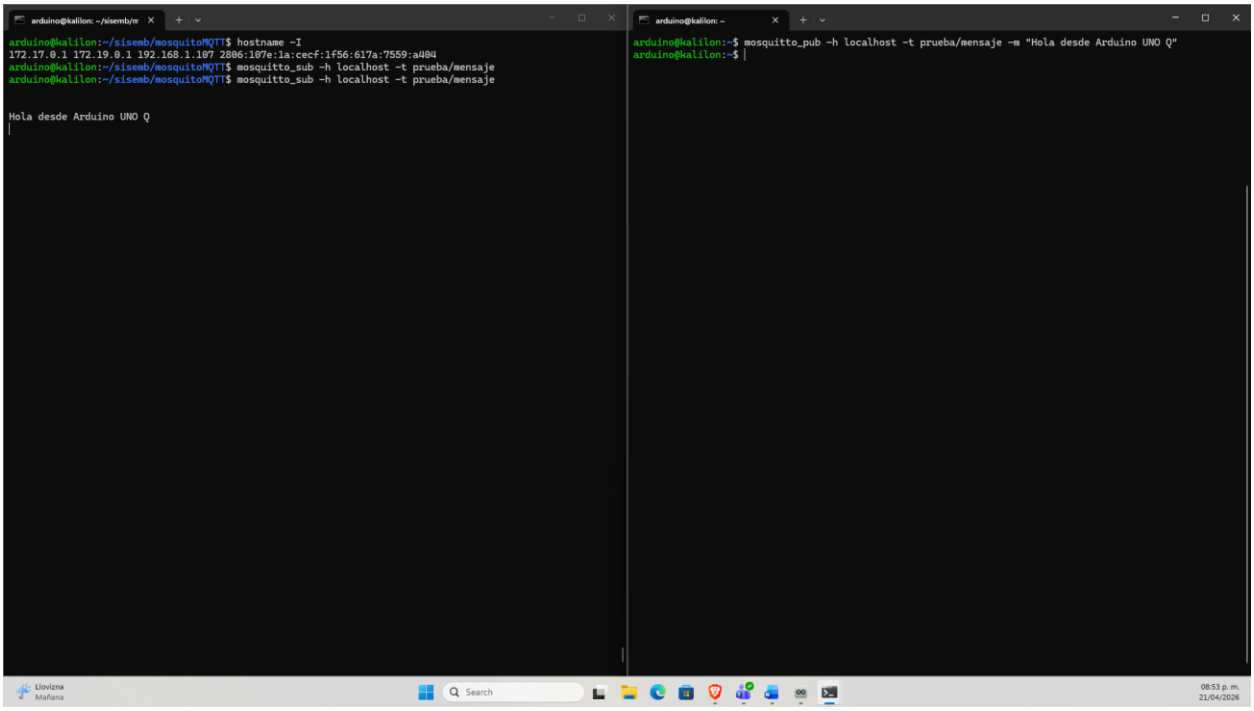


Ahora habilitamos el servicio para que arranque al iniciar nuestra Arduino



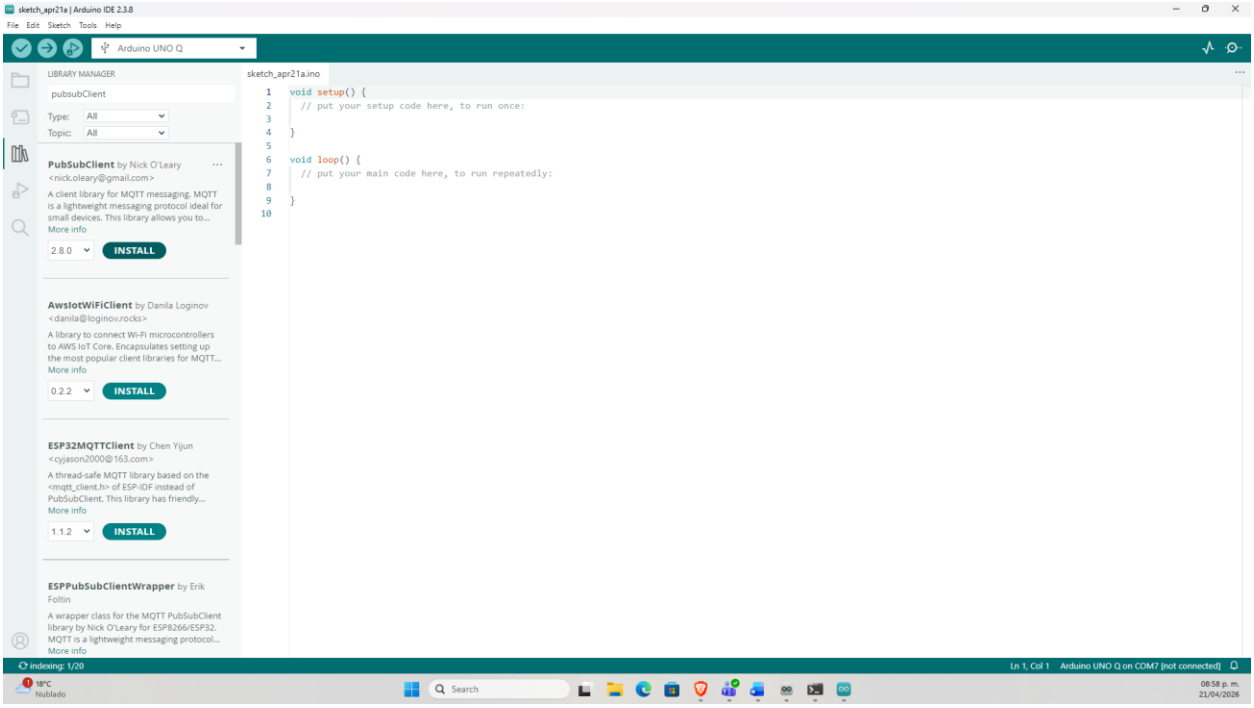
Prueba básica del bróker

Para hacer la prueba, primero identificamos cual es la dirección IP de nuestra Arduino, después ejecutaremos un subscriber, pero como estamos haciendo una prueba a la misma máquina, podemos colocar la IP o localhost, y desde otra consola de la misma maquina lanzamos el mensaje.



Configuración del Publisher en ESP32

Para programar nuestra ESP32 vamos a utilizar el entorno de Arduino IDE, y dentro de este IDE descargaremos la librería de PubSubClient.



En Nuestro servidor se deben hacer modificaciones para que este acepte conexiones remotas, así que modificamos el archivo de configuración.

```
arduino@kalilon: ~/sisemb/m
arduino@kalilon:~/sisemb/mosquitoMQTT$ sudo nano /etc/mosquitto/conf.d/default.conf
```

```
arduino@kalilon: ~/sisemb/m
  GNU nano 8.4                    /etc/mosquitto/conf.d/default.conf *
listener 1883
allow_anonymous true

^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location  M-U Undo      M-A Set Mark   M-] To Bracket  M-B Previous   ^_ Back        ^◀ Prev Word
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^B Where Was    M-F Next       ^▶ Forward     ^▶ Next Word
```

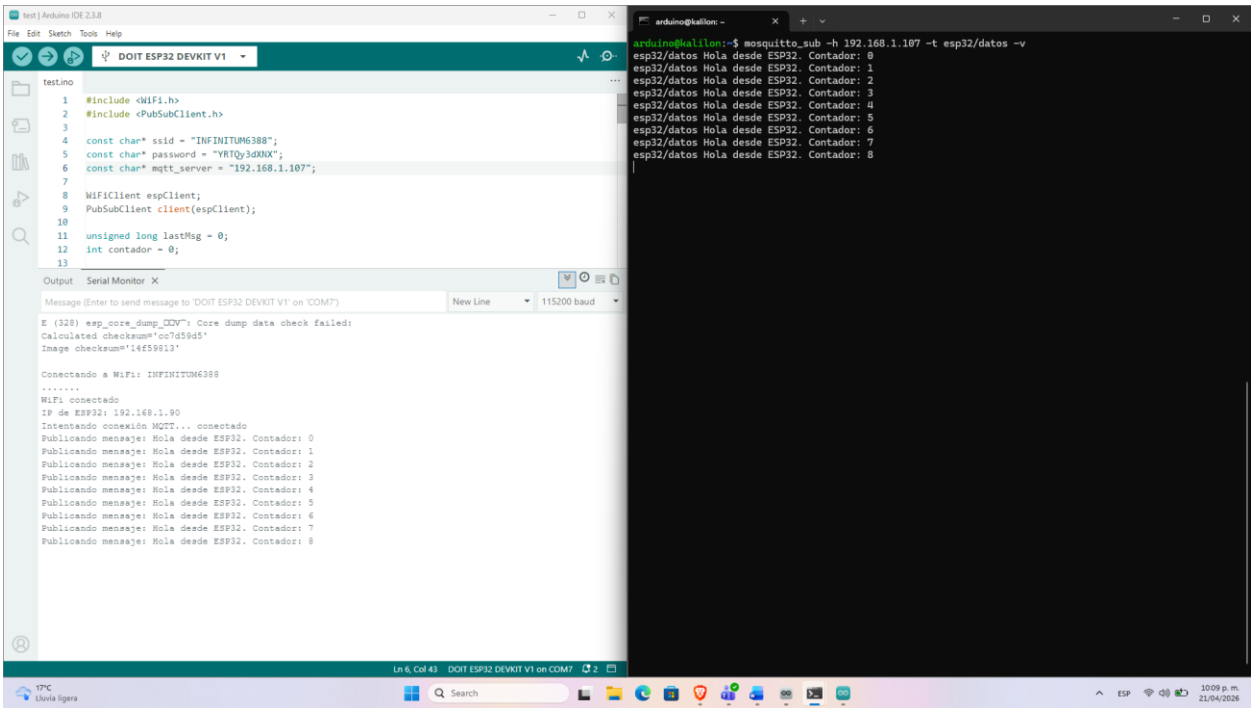
Despues de cambiar la configuracion debemos reiniciar el servicio

```
arduino@kalilon: ~
arduino@kalilon:~$ sudo systemctl restart mosquitto
[sudo] password for arduino:
arduino@kalilon:~$
```

Una vez que hemos descargado la librería, escribimos un código que mandará u mensaje a nuestro servidor MQTT cada 3 segundos, y el mensaje será un contador del total de mensajes que se han enviado, para este código necesitamos la dirección IP de nuestro servidor (IP de nuestro Arduino)

```
test.ino
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "INFINITUM6388";
const char* password = "YRTQy3dXNX";
const char* mqtt_server = "192.168.1.107";

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastMsg = 0;
int contador = 0;

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a WiFi: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi conectado");
  Serial.print("IP de ESP32: ");
```

```
Output   Serial Monitor
Writing at 0x000440cf [=====================>       ]  78.0% 114688/147119 bytes...

Writing at 0x0004c9aa [========================>    ]  89.1% 131072/147119 bytes...

Writing at 0x00052280 [==============================] 100.0% 147119/147119 bytes...
Wrote 270976 bytes (147119 compressed) at 0x00010000 in 2.4 seconds (888.6 kbit/s).
Verifying written data...
Hash of data verified.

Hard resetting via RTS pin...
```

Evidencia de funcionamiento

Ahora en nuestro Arduino escuchamos los mensajes y observamos que empiezan a llegar los mensajes desde nuestra ESP32